

Beginning Python

By: Dudi Kahana

Email: DudiK.Work@gmail.com

Phone: 050-9329352

Date: 2/26

Note: Do not use this document, without explicit permission from the author. ©

Python Introduction

History & Time line

- ☐ Invented in 1991.
- ☐ Python 1.0 releases in 1994.
- ☐ Python 2.0 released in 2000.
- ☐ Python 3.0 released in 2008
 - It is backward-incompatible.
- ☐ The current release is 3.14.x.

Features of Python

- ☐ High level language.
- ☐ Multi-paradigm: O.O.P, Procedural language.
- ☐ Simple and minimalistic, readable, easy to learn
- ☐ Interpreted.
- ☐ Automatic memory management.
- ☐ Cross platform.
- ☐ Large standard library.
- ☐ Download free, open source.

Compiler Vs. Interpreter (I)

☐ Compiler steps:

- Program Creation.
- Program analysis - throws errors in case of any incorrect statement.
- Converts the source code to machine code.
- Linking of various code files into a runnable program.
- Finally runs a Program.

☐ Interpreter steps:

- Program Creation.
- Execution of source statements one by one.

5

© by Dudi Kahana

Compiler Vs. Interpreter (II)

<u>Compiler</u>	<u>Interpreter</u>
Generates an output file (.exe), while off-line.	No off-line output generation.
Takes a large amount of time for analyzing the source code.	Take less time for analyzing the source code.
Compiled code run faster than Interpreter.	Interpreted codes run slower than Compiler.
It does not require source code for later execution.	It requires source code for later execution.
<u>Pass the all source at once:</u> - Errors are displayed for all code at once. - Code optimization in a global scope. - Any code change, after the compilation requires recompiling the entire code.	<u>Works source line by line:</u> - Errors are displayed for single line at once. - Code Optimization only for single line. - Any code change does not require retranslation of the entire code.
C, C++, C#, Java	Perl, Python, Matlab

6

© by Dudi Kahana

Compiler Vs. Interpreter (III)

	<u>Compiler</u>	<u>Interpreter</u>
Advantages	The executable runs faster.	Language is more simple & flexible.
	Helps security.	Easier to debug.
Disadvantages	Compilation takes more time for huge code.	Runs slower.
	Catch only syntax errors.	Run on computers that have the corresponding interpreter.
	Code change requires recompilation.	

7

© by Dudi Kahana

Static Vs. Dynamic type

<u>Static type</u>	<u>Dynamic type</u>
Requires explicit data-type declaration.	Does not require data-type declaration.
Data-types are checked at compile time.	Data-types are checked at run time.
Handle type errors at compilation.	Type errors may be unnoticed till run-time.
Languages: C, C++, C#, Java	Languages: Python, Perl.

8

© by Dudi Kahana

Strong Vs. Weak (loosely) type

How much type consistency is enforced in run-time

❑ Strong type:

- Guarantee accepted programs are type-safe.
- Languages: Java, C#, Python.

❑ Weak (loosely) type:

- Allow programs which contains type errors.
- Languages: C, C++.

Python Fundamentals

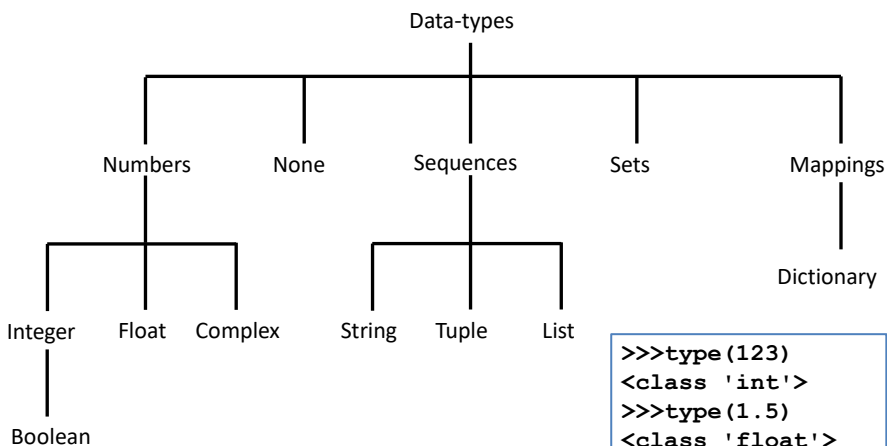
Python keywords

True	if	as	from
False	else	assert	import
None	elif	class	global
and	for	def	nonlocal
or	while	del	in
Not	continue	except	is
Return	Break	finally	lambda
Try	with	yield	pass
raise			

11

© by Dudi Kahana

Primitive data types



```
>>>type(123)
<class 'int'>
>>>type(1.5)
<class 'float'>
>>>type("Dudi")
<class 'str'>
>>>type([1,2,3])
<class 'list'>
```

12

© by Dudi Kahana

Variables (I)

- ❑ Variable must assigned – no default values.
- ❑ Variables are **dynamically typed**:
 - No need to declare type – only set a value.
 - The variable type can altered at any time.

```
>>>x = 324
>>>x
324
>>>type(x)
<class 'int'>
>>>x = 'Dudi'
>>>x
'Dudi'
>>>type(x)
<class 'str'>
```

13

© by Dudi Kahana

Variables (II)

- ❑ Variables are **strong typed**:
 - Any type mismatch causes a run-time error:

```
>>>45 + '789'
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    45 + '789'
TypeError: unsupported operand type(s) for +: 'int'
and 'str'
```

- Need to use type conversion:

```
>>>str(45) + '789'
'45789'
>>>45 + int('789')
834
```

14

© by Dudi Kahana

Logical Operators

		Binary Op.		Unary Op.
X	Y	X and Y	X or Y	not X
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

© by Dudi Kahana

15

Mathematical Operators

- Arithmetic: +, -, *, /, %, // (integer division), ** (pow).
- Assignment: =, +=, -=, *=, /=, %=, //=, **=.
- Comparison: <, >, <=, >=, ==, !=.
- Chained comparison: 1 <= x <= 100

□ Note:

- + on strings is strings concatenation.
- * on string is string repetition.

```
>>>999 / 10
99.9
>>>999 // 10
99
>>>str1 = "Hello"
>>>str2 = "World!"
>>>str1 + str2
'HelloWorld!'
>>>str1 * 4
'HelloHelloHelloHello'
>>>-3 ** 2
-9
>>> x=5; x=x+1
>>>print(x)
6
>>>x+=1; print(x)
7
>>>7
```

© by Dudi Kahana

16

Assignments

```
>>># Chained assignment
```

```
>>>x=y=z=1
>>>print(x, y, z)
1 1 1
```

```
>>># Simultaneously assignment
```

```
>>>x, y, z = 10, 11, 12
>>>print(x, y, z)
10 11 12
```

```
>>># Variables swapping
```

```
>>>y, x = x, y
>>>print(x, y, z)
11 10 12
```

```
>>># Sequence packing
```

```
>>>values = x, y, z
>>>values
(11, 10, 12)
```

```
>>># Sequence unpacking
```

```
>>>a, b, c = values
>>>a
11
>>>a, b, *rest = [21, 22, 23, 24, 25]
>>>rest
[23, 24, 25]
>>>a, *b, rest = [21, 22, 23, 24, 25]
>>>b
[22, 23, 24]
>>>a, b, rest
(21, [22, 23, 24], 25)
```

17

© by Dudi Kahana

Input and Output

```
>>>x = input("x: ")
x: 12
>>>y = input("y: ")
y: 4
>>>print(x, y)
12 4
>>>x, y
('12', '4')
>>>print(x * y)
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    print(x * y)
TypeError: can't multiply sequence by non-int of type 'str'
>>>print(int(x) * int(y))
48
```

18

© by Dudi Kahana

Boolean values

- ❑ Any object can be tested for truth value.
- ❑ False:
 - None($\equiv$ null).
 - False.
 - Zero of any numeric type.
 - Any empty sequence or dictionary: "", (), [], {}.
- ❑ True:
 - Everything else.

Memory layout(I)

Code line	Byte Content	Address
num = 4	0000 0100	1000
	0000 0000	1001
	0000 0000	1002
	0000 0000	1003
		1004
		1005

Left Nibble

Right Nibble

Memory layout(II)

Code line	Byte Content	Address
ch1 = 'd'	0110 0100	1000
num1 = 100	0110 0100	1001
ch2 = '9'	0011 1001	1002
num2 = 9	0000 1001	1003
		1004
		1005

Left Nibble

Right Nibble

© by Dudi Kahana

Flow-control

© by Dudi Kahana

Lines & Blocks

```
>>># Single line statement
```

```
>>>x = 5
>>>y = 15
>>>print(x + y)
20
```

```
>>># Multiple statements
```

```
>>># on a single line
>>>x = 5; y = 15; print(x+y)
20
```

```
>>> #Multi-line statement
```

```
>>>if 1<x<10 and 10<y<20 \
>>>    and 50<(x*y)<100:
```

```
>>>    print("correct numbers.")
>>>    print("correct product.")
```

```
correct numbers.
correct product.
```

```
-----
num_array = [1, 2, 3,
              4, 5, 6]
```

```
num_array
[1, 2, 3, 4, 5, 6]
```

Comment

Single statement

Code block

23

© by Dudi Kahana

Conditions

```
num = int(input("Enter a number: "))
if num > 0:
    print("The number is positive.")
elif num < 0:
    print("The number is negative.")
else:
    print("The number is zero.")

-----

sign = "positive" if num > 0 else "none positive"
print("The number is: ", sign)
```

24

© by Dudi Kahana

While Loops

```

num = 1489
tmp = num
sum = 0
while (tmp > 0):
    sum += tmp%10
    tmp //= 10
print("The digits sum of ", num, " is: ", sum)
-----
while (True):
    num = int(input("Enter a num(range 1-100, 0 to stop): "))
    if (num == 0):
        break # Stop looping.
    if (num > 100):
        print("Invalid value")
        continue # Skip to the start.
    # Process the read value...
    print("Valid value. Processing...")
# Code outside the loop...
print("The loop ended")

```

← While loop

← break, continue

25

© by Dudi Kahana

For Loops

```

for x in ['red', 'green', 'blue', 'yellow']:
    print(x)
-----
for x in range(10, 100):
    print(x)
-----
# Print the even numbers between 10-100.
for x in range(10, 100, 2):
    print(x)

```

26

© by Dudi Kahana

Function

Function name

Function argument(s)

```
def sum_of_digits(num):  
    digits_sum = 0  
    while num > 0:  
        digits_sum += num % 10  
        num //= 10  
    return sum
```

Function body

Function return value

```
num = int(input("Enter a number: "))  
result = sum_of_digits(num)  
print ("The digits sum of ", num, " is ", result)
```

Function call

27

© by Dudi Kahana

Sequences & Dictionaries

28

© by Dudi Kahana

Sequence types

❑ Consist of iterable objects.

❑ Sequence types:

- List - mutable, heterogeneous.
- Range - immutable homogeneous (only numbers).
- Tuple - immutable, heterogeneous.
- String - immutable homogeneous.

29

© by Dudi Kahana

List

❑ Examples:

- [] – Empty list.
- [6] – one element list.
- [6, -2, 1024] – many elements list.
- [5, 8.98, "Hello"] – heterogynous list.
- [{"Dudi", 34}, {"Elad", 25}, [{"Yaniv", 12}]] – list of lists.

❑ List initializations:

- L=[]
- L = [a] or L = [a, b, c]
- L = [x for x in range(10)]

30

© by Dudi Kahana

List Operations & Methods

❑ Examples:

```
>>>l = [None] * 5 #Init a list of length 5 with Null.
>>>l
[None, None, None, None, None]
>>>lang_name = list("Python") #Create list from string.
>>>lang_name
['P', 'y', 't', 'h', 'o', 'n']
>>>lang_name[1:] = list("erl") #Replace the end part elements.
>>>lang_name
['P', 'e', 'r', 'l']
>>>numbers = [1,5]
>>>numbers[1:1] = [2, 3, 4] #Insert a list into a list.
>>>numbers
[1, 2, 3, 4, 5]
```

❑ Learn & practice the following list methods:

· append · pop · clear · copy · extend
· insert · remove · reverse · sort

31

© by Dudi Kahana

List Comprehensions

❑ Examples:

```
>>> fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
>>> new_list_A = [x for x in fruits if "a" in x]
>>> print(new_list_A)
>>> ['apple', 'banana', 'mango']
>>> new_list_B = [x for x in fruits if x != "apple"]
>>> print(new_list_B)
>>> ['banana', 'cherry', 'kiwi', 'mango']
```

32

© by Dudi Kahana

Matrix - List Of lists

❑ Example:

```
matrix = [[4, 7, 1, 2],
          [2, 3, 5, 7],
          [8, 2, 5, 1]]

row_len = len(matrix)
column_len = len(matrix[0])

# Print matrix values
for i in range(row_len):
    print("")
    for j in range(column_len):
        print(matrix[i][j], " " , end = "")

# init matrix of size 3x4 to zero value.
matrix = [[0 for index in range(4)] for index in range(3)]
```

33

© by Dudi Kahana

Range

❑ Advantage over list: stores only start, stop & step.

❑ Examples:

```
>>>list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>>list(range(1, 11))
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>>list(range(0, 35, 7))
[0, 7, 14, 21, 28]
>>>list(range(0, 12, 5))
[0, 5, 10]
>>>list(range(0, -10, -1))
[0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
>>>list(range(0))
[]
>>>list(range(1, 0))
[]
```

34

© by Dudi Kahana

Tuple

❑Examples:

- () – Empty tuple.
- (6,) – one element tuple.
- (5, 8.98, "Hello") – heterogynous tuple.
- (("Dudi", 34), ("Elad", 25), ("Yaniv", 12)) – tuple of tuples.

❑Tuple initializations:

- T = ()
- T = (1,) or T = (2, 'b', 3)
- T = tuple() or T = tuple(iterable)

35

© by Dudi Kahana

Common Sequence Operations (I)

<u>Operation</u>	<u>Result</u>
x in S1	True if item of S1 is equal to x, otherwise False.
x not in S1	False if item of S1 is equal to x, otherwise True.
S1 + S2	concatenation of S1 and S2.
S1 * num	num shallow copies of S1.
S1[i]	i'th item of S1 from the left (first item is at i=0).
S1[-i]	i'th item of S1 from the right (last item is at i=-1).
S1[i:j]	slice of S1 from i to j (the j index is after the last item). Negative numbers counts from the end.
S1[i:j:k]	slice of S1 from l to j with step k.
len(S1)	Length of S1.

36

© by Dudi Kahana

Common Sequence Operations (II)

<u>Operation</u>	<u>Result</u>
<code>min(S1)</code>	smallest item of S1.
<code>max(S1)</code>	largest item of S1.
<code>S1.index(x)</code>	index of the first occurrence of x in S1.
<code>S1.count(x)</code>	total number of occurrence of x in S1.

❑ Example:

```
def is_contained(s1, s2):
    for i in s2:
        if i not in s1:
            return False
    return True
```

37

© by Dudi Kahana

String

❑ Examples:

```
word = 'Python'
sentence = "I love Python"
paragraph = '''This is a paragraph,
it consists of multiple lines.
This is better than C language.'''
```

❑ Learn & practice the following string methods:

- `find()` · `join()` · `isupper()` · `islower()`
- `lower()` · `replace()` · `isalpha()`
- `strip()` · `split()` · `isdigit()`

For more methods see:

<https://www.programiz.com/python-programming/methods/string>

38

© by Dudi Kahana

ASCII Conversions

ord / chr functions:

```
str = "Hello, World!"
first_char = str[0]

ascii_value = ord(first_char)
print("The ASCII value of {} is: {}".format(first_char,
                                             ascii_value))

first_char = chr(ascii_value)
print("The char of {} value is: {}".format(ascii_value,
                                             first_char))
```

39

© by Dudi Kahana

String Formatting

```
format = "The product: %s | The price: %s"
values = ("Bread", "14.90")
format % values
'The product: Bread | The price: 14.90'

-----

from string import Template
templ = Template("The product: %s | The price: %s")
templ = Template("The product: $name | The price: $price")
templ.substitute(name = "Bread", price = "14.90" )
'The product: Bread | The price: 14.90'

-----

"The product: {} | The price: {}".format("Bread", "14.90")
'The product: Bread | The price: 14.90'

-----

"The product: {name} | The price: {value:.1f}".format(name=
                                                    "Bread", value=14.90)
'The product: Bread | The price: 14.9'
```

40

© by Dudi Kahana

Dictionary

- ☐ Hash-tables – consist of key-value pairs.
- ☐ Complexity – $O(1)$
- ☐ Key must be immutable type. Value can be any type.
- ☐ Dictionary initialization:

```
>>>my_dictionary = {'one':1, 'two':2, 'three':3}
>>>a = dict(one=1, two=2, three=3)
>>>b = dict(zip(['one', 'two', 'three'], [1,2,3]))
>>>c = dict([('two', 2), ('one', 1), ('three', 3)])
>>>my_dictionary == a == b == c
True
```

- ☐ Learn & practice the following dictionary methods:

· update · pop · copy · clear
· get · keys · values · loop over keys

41

© by Dudi Kahana

Set

- ☐ Unordered, unchangeable, unindexed.
- ☐ Heterogeneous, uniqueness, complexity – $O(1)$
- ☐ Math operations: intersection() → $\&$, union() → $|$, difference() → $-$, symmetric_difference → $\wedge$.
- ☐ Set initialization:

```
>>>fruits = {"apple", "banana", "cherry"}
>>>fruits
>>> {"apple", "banana", "cherry"}
```

- ☐ Learn & practice the following set methods:

· add · pop · copy · clear
· remove · loop over items

42

© by Dudi Kahana

Recursion (I)

❑ What is recursion?

- The method calls itself.
- Solving the high order case, using the simple case.

```
def star(num):
```

```
    if num < 1:
        return
```

```
    star(num - 1)
```

```
    print("* ")
```

Recursion parts:

I. Stop condition

II. Recursive call

III. Doing the job

© By Dudi Kahana

Recursion (II)

❑ Head recursion:

```
def star(num):
    if num < 1:
        return
    star(num - 1)
    print(num)
```

❑ Tail recursion

```
def star(num):
    if num < 1:
        return
    print(num)
    star(num - 1)
```

© By Dudi Kahana

Recursion Examples

❑ Factorial:

$0! = 1, 1! = 1, 2! = 2, 3! = 6, 4! = 24, \dots$

```
def factorial(num):
    if num <= 1:
        return 1
    return num * factorial(num-1)
```

❑ Fibonacci:

$$a_0=0$$

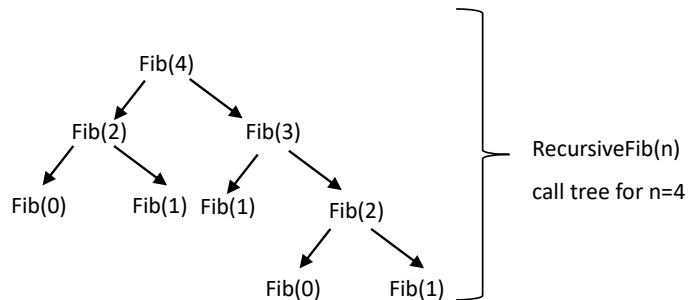
$$a_1=1$$

$$a_n = a_{n-1} + a_{n-2}$$

0,1,1,2,3,5,8,...

© By Dudi Kahana

Recursive Fibonacci Call Tree



© By Dudi Kahana

Complexity

- ❑ How much does it cost?
- Time complexity - $T(n)$.
 - Memory complexity - $P(n)$.
 - Measured as function of the input size (why?).
 - Examples for input size:
 - Array – number of cells.
 - String – number of characters.
 - Tree – number of nodes.

© By Dudi Kahana

Upper Bound

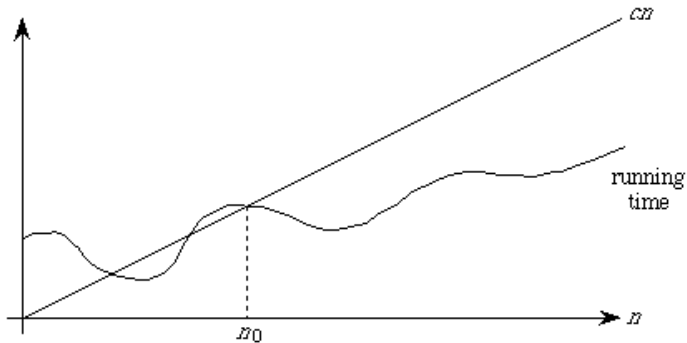
$$T(n) = 0.01n^2 + 10n + 1000$$

<i>n</i>	<i>a</i>	<i>b</i>	<i>c</i>
10	1	100	1K
100	100	1000	1K
1000	10K	10K	1K
10K	1M	100K	1K
100K	100M	1M	1K

- Assuming n is a very huge number – **a** is dominant.
- $T(n) = O(n^2)$

© By Dudi Kahana

Upper Bound (another view)



- Asymptotic upper bound.
- Running time = $T(n) = O(n)$

© By Dudi Kahana

Worst case secenario

❑ Three scenarios for the same input size n :

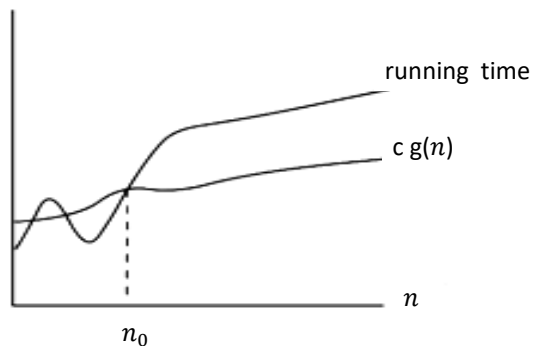
- Best case
- Average case
- Worst case

An example: Checking if a given number exists in a given vector.

❑ The asymptotic upper bound is over the worst-case.

© By Dudi Kahana

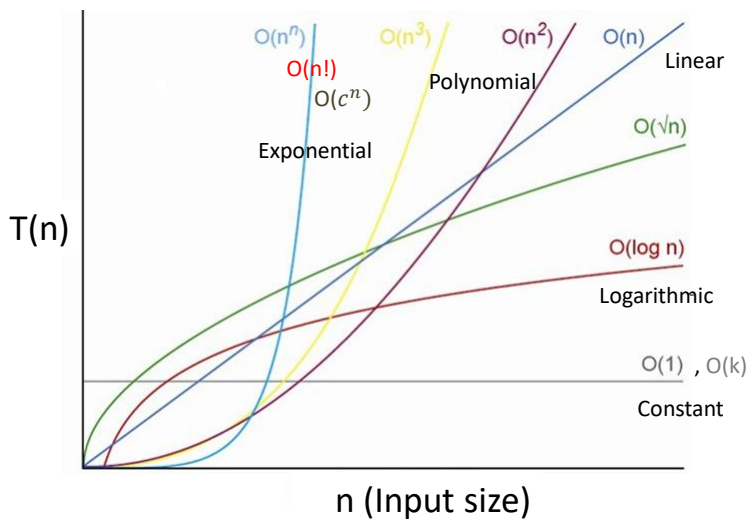
More bounds



- Asymptotic lower bound.
- $T(n) = \Omega(g(n))$
- When $T(n) = \Omega(g(n))$ and $T(n) = O(g(n))$ it is tight asymptotic bound: $T(n) = \theta(g(n))$

© By Dudi Kahana

Time Complexity



© By Dudi Kahana

Type of improvements

- ❑ Checking if a given integer n is a prime number

Three algorithms:

- 1) Divide it by the numbers: $2.. n - 1$ - $O(n)$
The naïve algorithm
- 2) Divide it by the numbers: $2.. n/2$ - $O(n)$
Improvement by factor.
- 3) Divide it by the numbers: $2.. \sqrt{n}$ - $O(\sqrt{n})$
Magnitude of order improvement.

© By Dudi Kahana

Object Oriented Programing (O.O.P)

54

© by Dudi Kahana

O.O.P – Principles

❑ Encapsulation:

- Bundling data and methods into a single unit.
- Information hiding – privacy.

❑ Inheritance

❑ Polymorphism

} Learn in O.O.P course

© By Dudi Kahana

Class parts

Class MyCounter: ← Class name (PascalCase convention)

Constructor

```
def __init__(self, start):
    self.count = start
```

Class property

Object reference

Methods

```
def up(self, n=1):
    self.count += n
```

```
def down(self, n=1):
    self.count -= n
```

56

© by Dudi Kahana

Creating object

```
a = MyCounter(10)
a.up()
print(a.count)
a.down()
print(a.count)
```

Output:
11
10

57

© by Dudi Kahana

Hidden data

```
Class MyCounter:
    def __init__(self, start):
        self.__count = start
    def set_counter(self, n=1):
        self.count = n
    def get_counter(self):
        return self.count
```

58

© by Dudi Kahana